Week 2, Day 4: JPA, Hibernate and Linked Lists

Prepared for Upendra Kumar Sharma | 3–4 hours | Main topic: JPA/Hibernate, N+1, lazy vs eager, entity states, caching | DSA: Linked lists

Today's Schedule

- **0:00–0:40** Study the JPA stack, persistence context and entity states (sections 1 and 2).
- **0:40–1:20** Study relationships, fetching and the N+1 problem (sections 3 and 4). Reproduce N+1 in a small project and watch the SQL.
- **1:20–1:50** Study caching and performance tips (sections 5 and 6).
- **1:50–2:50** DSA: solve the 3 linked list problems (section 7). Spend at most 25 minutes per problem before reading the hint.
- **2:50–3:20** Answer the interview questions aloud (section 8) and add 10 flashcards.

1. The JPA Stack and the Persistence Context

Who is who

- **JPA (Jakarta Persistence):** the specification (annotations and the EntityManager API). In Spring Boot 3 the package is **jakarta.persistence**, not javax.
- **Hibernate:** the most common JPA implementation, which Spring Boot uses by default (Hibernate 6 with Boot 3).
- **Spring Data JPA:** a layer on top that generates repository implementations from interfaces, so you write findByStatus(...) instead of boilerplate.

Persistence context (first-level cache)

- The EntityManager (Hibernate calls it a Session) keeps a **persistence context**: a map of the entities it currently manages. In a Spring app it normally lives for the duration of one transaction.
- **Identity guarantee:** loading the same row twice in one persistence context returns the **same Java object**, with no second query for find-by-id.
- **Dirty checking:** Hibernate keeps a snapshot of each managed entity. At **flush** time it compares and issues UPDATE statements for changes, so you do not need to call save() on a managed entity.

- **Flush:** happens automatically before a query that might be affected, and at commit. flush() sends SQL to the database but does not commit.
- **Write-behind:** SQL is not sent immediately on persist() or on setters. It is queued and sent at flush, which allows batching.

2. Entity States and Operations

The four states

- **Transient (new):** created with new, not known to any persistence context, no database row.
- **Managed (persistent):** attached to a persistence context. Changes are tracked and saved at flush.
- **Detached:** was managed, but the persistence context closed or the entity was detached. Changes are no longer tracked.
- **Removed:** scheduled for deletion at flush.

Operations

- **persist(entity):** transient to managed. Schedules an INSERT.
- **merge(entity):** copies the state of a detached (or new) entity onto a managed instance and **returns the managed copy**. The object you passed in stays detached.
- **remove(entity):** managed to removed.
- **find(id):** loads by id, hitting the database if not already managed.
- **getReference(id):** returns a lazy proxy without querying. Spring Data calls it getReferenceById. Useful for setting a foreign key without loading the whole row.
- **detach() / clear():** managed to detached.
- **Spring Data save():** calls persist if the entity is new (id is null or the @Version is null) and merge otherwise. For an already managed entity inside a transaction, you do not need it.

ID generation

- **IDENTITY:** the database auto-increments. Simple, but Hibernate must insert immediately to learn the id, which **disables JDBC batch inserts**.
- **SEQUENCE:** uses a database sequence. The preferred choice on PostgreSQL, and it supports batching and pooled allocation.
- **UUID:** good for distributed systems and exposing ids publicly, but larger indexes.

3. Relationships and Mapping

- **@ManyToOne** is the most common and usually the owning side, holding the foreign key column (@JoinColumn).
- **@OneToMany(mappedBy = ...)** is the inverse side of a bidirectional relationship. The **owning side is the one without mappedBy**, and only the owning side's changes are written to the database.
- **@ManyToMany** needs a join table. For anything with extra columns, model the join table as its own entity with two @ManyToOne associations.
- **@OneToOne** can be tricky to make lazy on the inverse side. Use a shared primary key (@MapsId) when possible.
- **Cascade:** CascadeType.PERSIST, MERGE, REMOVE, ALL. Cascade down from parent to owned children only. Never cascade REMOVE across independent entities.
- **orphanRemoval = true:** deletes a child removed from the parent's collection.
- **Keep both sides in sync** with helper methods, since JPA does not do it for you.
- **Collections:** prefer Set for @ManyToMany. A List without an order column behaves as a "bag", and removing an element can delete and re-insert all rows.
- **equals and hashCode on entities:** be careful. Use a stable business key, or an id-based implementation that handles the null-before-persist case. Do not include lazy collections.

```
@Entity
public class Author {
    @Id @GeneratedValue(strategy = GenerationType.SEQUENCE)
    private Long id;
    private String name;

    @OneToMany(mappedBy = "author", cascade = CascadeType.ALL, orphanRemoval
= true)
    private List<Book> books = new ArrayList<>();

    public void addBook(Book book) {      // keeps both sides in sync
        books.add(book);
        book.setAuthor(this);
    }
}

@Entity
public class Book {
    @Id @GeneratedValue(strategy = GenerationType.SEQUENCE)
    private Long id;
    private String title;
```

```
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "author_id")
    private Author author;
}
```

4. Lazy vs Eager Loading and the N+1 Problem

Fetch defaults

- **To-one associations** (@ManyToOne, @OneToOne): **EAGER** by default.
- **To-many associations** (@OneToMany, @ManyToMany): **LAZY** by default.
- **Best practice:** set every association to **LAZY** and fetch what you need per use case with a query. EAGER cannot be switched off for a particular query, and it is a leading cause of N+1.

How lazy loading works: Hibernate gives you a proxy (for to-one) or a special collection (for to-many). The SQL runs the first time you touch it, and it needs an **open session**. Touching it after the transaction ends causes **LazyInitializationException**.

The N+1 problem

You run **1 query** to load N parents, and then Hibernate runs **N more queries**, one per parent, as you access each parent's association.

```
List<Author> authors = authorRepository.findAll();       // 1 query
for (Author a : authors) {
    System.out.println(a.getBooks().size());             // N extra queries
}
```

With 1,000 authors that is 1,001 round trips to the database. It is invisible in development with small data and a disaster in production.

How to detect it

- Turn on SQL logging (logging.level.org.hibernate.SQL=DEBUG) and count the statements for one request.
- Enable Hibernate statistics (hibernate.generate_statistics=true) and check the query count.
- Use a tool such as p6spy or datasource-proxy, and write tests that assert the number of queries.

Fixes, from most to least targeted

1. **JOIN FETCH** in JPQL loads the parents and the association in one query.

2. **@EntityGraph** on a repository method does the same without writing JPQL.
3. **DTO projections** select only the columns you need into a record or interface. Often the best choice for read endpoints, because you skip entity management entirely.
4. **Batch fetching:** @BatchSize or hibernate.default_batch_fetch_size=50 loads lazy associations for many parents with one IN query (1 + N/50 queries instead of 1 + N).
5. **SUBSELECT fetch mode** (@Fetch(FetchMode.SUBSELECT)) loads all collections for the parent query's result in one extra query.

```
public interface AuthorRepository extends JpaRepository<Author, Long> {

    @Query("select distinct a from Author a join fetch a.books")
    List<Author> findAllWithBooks();

    @EntityGraph(attributePaths = "books")
    List<Author> findByNameStartingWith(String prefix);

    @Query("select new com.example.AuthorSummary(a.id, a.name, count(b)) " +
           "from Author a left join a.books b group by a.id, a.name")
    List<AuthorSummary> findSummaries();
}

public record AuthorSummary(Long id, String name, Long bookCount) {}
```

Fetch-join gotchas

- **Pagination with a collection fetch:** Hibernate cannot apply LIMIT in SQL, so it paginates **in memory** and logs a warning (HHH000104). Fix it by paging the parent ids first and fetching associations in a second query, or by using batch fetching.
- **Fetching two List collections at once** throws MultipleBagFetchException. Use Sets, or fetch them in separate queries.
- **Duplicate parents:** a join fetch on a collection returns one row per child, so use distinct (Hibernate 6 removes duplicates automatically, but be aware on older versions).

5. Caching

- **First-level cache:** the persistence context. Always on, one per session, not shared between threads or transactions.
- **Second-level cache:** optional, shared across sessions at the SessionFactory level, caches entities by id (and collections or queries if enabled). It needs a provider such as Ehcache or Caffeine through JCache, and you mark entities with @Cache or @Cacheable. Concurrency strategies include READ_ONLY, NONSTRICT_READ_WRITE, READ_WRITE and TRANSACTIONAL.

- **Query cache:** caches query result ids. Invalidated whenever the underlying tables change, so it rarely pays off.
- **Reality check:** many teams skip the second-level cache and instead cache at the application level with Spring Cache and Redis, which is easier to reason about and works across service instances. Be ready to discuss staleness and invalidation either way.

6. Performance Tips You Can Mention in Interviews

- Make all associations LAZY, and fetch with join fetch, entity graphs or DTO projections.
- **Paginate** (Pageable) instead of loading everything. Never call findAll() on a large table.
- Use **readOnly = true** on query-only transactions to skip dirty checking.
- Enable JDBC batching for bulk writes: hibernate.jdbc.batch_size=50, order_inserts and order_updates, and use SEQUENCE ids.
- Use **@Modifying** bulk queries for large updates. They bypass the persistence context, so use clearAutomatically = true or reload afterwards.
- Add the right indexes for the columns you filter, join and sort on (Day 5).
- **Open-in-view:** Spring Boot enables it by default, which keeps the session open through the web layer and can hide N+1 and hold connections longer. Many teams set spring.jpa.open-in-view=false.
- **Avoid exposing entities directly** from controllers. Map to DTOs, which avoids lazy-loading surprises, accidental data leaks and infinite recursion in JSON.
- Check the SQL Hibernate generates for your hottest endpoints.

7. DSA: Linked Lists (3 Problems)

All three use this node class.

```
public class ListNode {
    int val;
    ListNode next;
    ListNode(int val) { this.val = val; }
}
```

Problem 1: Reverse Linked List

- **Idea (iterative):** walk the list with three pointers. For each node, save its next, point it at the previous node, then advance both pointers.

- **Complexity:** O(n) time, O(1) space. The recursive version is also O(n) time but uses O(n) stack space.
- **Edge cases:** empty list, single node.

```
public ListNode reverseList(ListNode head) {
    ListNode prev = null, curr = head;
    while (curr != null) {
        ListNode next = curr.next;
        curr.next = prev;
        prev = curr;
        curr = next;
    }
    return prev;
}
```

Problem 2: Merge Two Sorted Lists

- **Idea:** use a **dummy head** node so you never have to special-case the first element. Repeatedly attach the smaller of the two current nodes, then attach whatever remains.
- **Complexity:** O(n + m) time, O(1) extra space.
- **Edge cases:** one or both lists empty, all elements of one list smaller than the other, equal values.

```
public ListNode mergeTwoLists(ListNode a, ListNode b) {
    ListNode dummy = new ListNode(0), tail = dummy;
    while (a != null && b != null) {
        if (a.val <= b.val) { tail.next = a; a = a.next; }
        else                { tail.next = b; b = b.next; }
        tail = tail.next;
    }
    tail.next = (a != null) ? a : b;
    return dummy.next;
}
```

Problem 3: Linked List Cycle

- **Idea:** Floyd's tortoise and hare. Move one pointer one step and another two steps. If they ever meet, there is a cycle. If the fast pointer reaches null, there is none.
- **Complexity:** O(n) time, O(1) space. A HashSet of visited nodes also works but uses O(n) space.
- **Edge cases:** empty list, single node pointing to itself, a cycle that starts at the head.

- **Follow-up to prepare (Linked List Cycle II):** after the pointers meet, reset one pointer to the head and move both one step at a time. They meet at the node where the cycle begins.

```
public boolean hasCycle(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) return true;
    }
    return false;
}
```

8. Interview Questions to Answer Aloud

1. What is the difference between JPA, Hibernate and Spring Data JPA?
2. What is the persistence context, and what does the first-level cache guarantee?
3. What are the four entity states, and how do persist, merge and remove move entities between them?
4. How does dirty checking work, and why do you not need save() on a managed entity?
5. What is the difference between persist and merge? What does Spring Data's save() do?
6. Which side owns a bidirectional relationship, and what does mappedBy mean?
7. What are the default fetch types, and why is LAZY recommended everywhere?
8. What is the N+1 problem? How do you detect it and fix it?
9. JOIN FETCH vs @EntityGraph vs DTO projection vs batch fetching: when would you pick each?
10. What goes wrong when you paginate with a collection join fetch?
11. What causes LazyInitializationException, and what is wrong with fixing it by switching to EAGER?
12. Explain the first-level, second-level and query caches. Would you use the second-level cache?
13. Why does IDENTITY generation prevent JDBC batch inserts, and what do you use instead on PostgreSQL?

9. End-of-Day Checklist

- [ ] I can explain the persistence context, dirty checking and the four entity states.

- [ ] I can explain owning vs inverse side, cascade and orphanRemoval.
- [ ] I reproduced an N+1 problem and fixed it with join fetch, an entity graph and a DTO projection.
- [ ] I can explain the pagination-with-fetch-join trap and its fix.
- [ ] I can compare the first-level, second-level and application-level caches.
- [ ] I solved all 3 linked list problems and can explain Floyd's algorithm, including the cycle start follow-up.
- [ ] I wrote at least 10 flashcards from questions I got wrong.

Tomorrow (Week 2, Day 5): SQL indexing, B-trees, EXPLAIN ANALYZE, joins, locking and pagination, plus SQL practice problems.